



UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad
Intelectual (Real Decreto Ley 1/1996 de 12 de abril).
Queda expresamente prohibido su uso o distribución
sin autorización del autor.

Tecnologías Web

3º Grado en Ingeniería Informática

Guión de prácticas El lenguaje HTML

1. Objetivo.....	2
Sesión 1.....	3
2. El estándar HTML.....	4
3. La estructura de una página web.....	5
4. Codificación de los caracteres.....	6
5. Tablas en HTML.....	8
Sesión 2.....	10
6. Formularios.....	11
7. Sitios web.....	17
8. Validación de documentos HTML.....	19
9. Entrega de la práctica.....	20

© Prof. Javier Martínez Baena
Dpto. Ciencias de la Computación e I. A.
Universidad de Granada



Departamento de
Ciencias de la Computación
e Inteligencia Artificial

1. Objetivo

El objetivo de la práctica es desarrollar algunas páginas web sencillas con HTML haciendo especial hincapié en el uso correcto de las etiquetas estructurales del lenguaje. Tenga en cuenta que:

1. No debe preocuparse del aspecto de la página final.
2. Debe centrarse únicamente en redactar código HTML.
3. No puede usar código para dar formato (CSS) **en ningún sitio** salvo, quizá, en aquellos puntos en donde se indique de forma explícita.

Aunque no use elementos de estilo (CSS), es importante que en cada ejercicio tenga un diseño de la página final que resultaría si le aplicase estilo. Esto es importante porque puede que condicione algunos aspectos de la estructura del documento HTML o, al menos, le puede ayudar a definir mejor la estructura del documento.

El diseño previo, aunque puede hacerlo en papel, sería más adecuado hacerlo usando alguna herramienta software que le ayude como, por ejemplo, Pencil.

Esta práctica solo requiere implementar usando el lenguaje HTML y, por lo tanto, no necesita un servidor web. Es decir, podría hacer la implementación en su equipo (local) y visualizar el documento con su navegador. Sin embargo, para probar los formularios, se pide que incluya un pequeño *script* PHP y para esto sí necesita un servidor web funcionando: puede usar `void.ugr.es` o su propio servidor (instalado en su máquina) pero asegúrese de que la página funciona correctamente cuando la suba a `void.ugr.es`.

A partir de esta práctica se asume que tiene instalado y configurado correctamente un servidor web Apache+PHP en su puesto de trabajo así como las herramientas de desarrollo de su elección (IDE, navegador, etc.). Si no es así deberá finalizar con éxito las prácticas anteriores antes de continuar.

Puede utilizar su propio ordenador personal o, si no dispone de él, el PC del aula de prácticas con el código de arranque y credenciales siguientes:

- Código de arranque: `ubutwebvi`
- Usuario: `aulas`
- Clave: `aulas`

Sesión 1

Introducción a HTML

2. El estándar HTML

El uso de *tags* para la creación de documentos HTML puede parecer una tarea simple a primera vista. La dificultad estriba en hacerlo de la mejor forma posible. A poco que leamos el estándar de HTML vemos que la dificultad no reside en la sintaxis de escritura de los *tags* sino en los matices y condicionantes a tener en cuenta en cada uno de ellos. Por tanto, para crear un documento HTML es necesario:

- Tener un buen conocimiento del estándar (<https://html.spec.whatwg.org/multipage/>).
- Elegir el *tag* más adecuado para cada elemento de la página en base a la semántica del mismo más que en relación al aspecto que nos gustaría que tuviese.
- Agrupar dichos *tags* y crear una jerarquía adecuada que estructure la información convenientemente y facilite el diseño posterior con CSS.

► Ejercicio 1: Crear una página básica

Redacte el código HTML de la siguiente página:

Charles Babbage

Charles Babbage fue un matemático y científico británico. Diseñó y desarrolló una calculadora mecánica capaz de calcular tablas de funciones numéricas por el método de diferencias. Publicó la obra *On the Economy of Machinery and Manufactures* [Sobre la economía de maquinaria y manufacturas] en donde afirmaba que:

Dividiendo el trabajo total en diferentes procesos, cada uno de los cuales requiere diferentes grados de destreza o de fuerza, el patrón puede comprar exactamente la cantidad precisa de ambas (fuerza y destreza) que es necesaria para cada proceso.

Algunos logros

- 1828-1839. Fue profesor de matemáticas en [Cambridge](#).
- 1833-1842. Diseñó la máquina analítica programable.
- Inventó el apartavacas, un aparato que se sujetaba a la parte delantera de las locomotoras de vapor para que las vacas se apartasen de las vías del ferrocarril.

¿Quieres saber más?

1. **Charles Babbage**. (1832). *On the economy of machinery and manufactures*. Cambridge University Press. ISBN 9780511696374
2. **Charles Babbage**. (1889). *Babbage's calculating engines*. Cambridge University Press. ISBN 9780511694721
3. **Lucy May Simister**. (2015). *Charles Babbage from the Beginning*. CreateSpace Independent Publishing Platform. ISBN 978-1512149197

Puede usar el siguiente texto como base del ejercicio:

Charles Babbage

Charles Babbage fue un matemático y científico británico. Diseñó y desarrolló una calculadora mecánica capaz de calcular tablas de funciones numéricas por el método de diferencias. Publicó la obra *On the Economy of Machinery and Manufactures* [Sobre la economía de maquinaria y manufacturas] en donde afirmaba que:

Dividiendo el trabajo total en diferentes procesos, cada uno de los cuales requiere diferentes grados de destreza o de fuerza, el patrón puede comprar exactamente la cantidad precisa de ambas (fuerza y destreza) que es necesaria para cada proceso.

Algunos logros

1828-1839. Fue profesor de matemáticas en Cambridge.

1833-1842. Diseñó la máquina analítica programable.

Inventó el apartavacas, un aparato que se sujetaba a la parte delantera de las locomotoras de vapor para que las vacas se apartasen de las vías del ferrocarril.

¿Quieres saber más?

Charles Babbage. (1832). *On the economy of machinery and manufactures*. Cambridge University Press. ISBN 9780511696374

Charles Babbage. (1889). *Babbage's calculating engines*. Cambridge University Press. ISBN 9780511694721

Lucy May Simister. (2015). *Charles Babbage from the Beginning*. CreateSpace Independent Publishing Platform. ISBN 978-1512149197

3. La estructura de una página web

Otra de las dificultades implícitas en el desarrollo de páginas web está en conseguir hacer una separación de conceptos clara entre información+estructura y aspecto o diseño de la página.

► Ejercicio 2: Inferir la estructura de una página web

En la siguiente figura puede ver la página web de una administración pública:

The screenshot shows the official website of the Junta de Andalucía, specifically the 'Consejería de Desarrollo Educativo y Formación Profesional'. The page layout includes a top header with the logo and navigation links (Alumnado, Profesorado, Familias, Centros, Actualidad). Below this is a large banner for 'Andalucía te conecta' featuring a woman on a smartphone and a man shaking hands, with text about digital capacity courses. To the right is a sidebar with a menu of services like 'Oficina virtual', 'Escolarización', 'Becas y ayudas', etc. The main content area below the banner lists 'Novedades' (News) with several announcements dated December 2023. The footer contains social media links, a directory of links, and a map of the region.

A partir de ella, debe inferir cual es la estructura del documento HTML. Es decir, la jerarquía de tags más adecuada para presentar la información en esa página. Tenga en cuenta que no hay una solución única para este ejercicio.

No tenga en cuenta detalles de aspecto o contenidos concretos. Céntrese en redactar la estructura del sitio prestando atención a la jerarquía de los elementos que aparecen y al uso de elementos estructurales de HTML.

3.1. Jerarquía HTML

Suponga que la Consejería con competencias en educación del ejercicio anterior quiere diseñar una página con un listado completo de centros educativos de la comunidad autónoma. En particular, para cada centro quiere mostrar:

- Nombre del Centro.
- Email del centro.
- Niveles educativos impartidos (primaria, secundaria, etc.).

En una aplicación real lo lógico sería disponer de algún tipo de organización que muestre listados parciales por localidad, por provincia, por tipo de centro, etc. pero nosotros vamos a diseñar la estructura de una única página web que contenga todos los centros andaluces.

► Ejercicio 3: Listado de centros educativos

Cree una página web con un listado de todos los centros educativos de Andalucía. ¡No se asuste! ... en este ejercicio solo debe incluir algunos a modo de ejemplo. Para realizar este ejercicio tenga en cuenta las siguientes indicaciones:

- Para cada centro se incluirá esta información: nombre del centro, email, niveles educativos.
- Los centros estarán agrupados por localidades.
- Las localidades estarán agrupadas por provincia.

Lo importante de este ejercicio es la creación de una jerarquía de *tags* que se corresponda con la estructura de la información en los términos indicados (básicamente en términos geográficos).

Si la estructura es correcta, más adelante se podrán crear diferentes diseños con CSS para presentarla de formas variadas pero siempre manteniendo la coherencia de la estructura original de la información.

Para que el ejemplo quede claro deberá incluir, al menos:

- 2 provincias.
- 3 poblaciones en cada provincia.
- 3 centros por población.
- Use datos de centros reales a partir de la web:
<https://www.juntadeandalucia.es/educacion/vscripts/centros/>.

► Ejercicio 4: Índice de centros

Cree una segunda página web con un listado alfabético de todos los centros educativos del ejercicio anterior con enlaces al fragmento de la primera página en donde se encuentra la información de cada centro. De esta forma, cuando clicamos sobre el nombre de un Centro, el navegador saltará a la página del ejercicio anterior y se posicionará sobre el Centro elegido.

4. Tablas en HTML

El uso de tablas en HTML debe limitarse a situaciones en las que la información que se va a presentar tiene de forma intrínseca una estructura tabular. En ningún caso deben utilizarse pensando en el maquetado de una página. Un caso típico de uso de una tabla es cuando mostramos un listado de tuplas obtenidas desde una BBDD.

► Ejercicio 5: Tabla de usuarios

Cree una tabla para mostrar la lista de usuarios de un sistema web con el aspecto de la siguiente figura:

Usuarios del sistema			
	Pepito Pérez de los Pérez y Peritos	Administrador	
		12345678Z	
	pepito@pepserver.com	<i>Editar</i>	<i>Borrar</i>
	Antonio Javier de Peláez y Peláez	Usuario normal	
		55555555S	
	antonio@pepserver.com	<i>Editar</i>	<i>Borrar</i>
	Mariquilla Martínez Martínez	Usuario normal	
		11111111A	
	maria@pepserver.com	<i>Editar</i>	<i>Borrar</i>

Sesión 2

Formularios y sitio web completo

5. Formularios

El uso de formularios es algo extremadamente usual en cualquier aplicación web. Estos nos permiten enviar información al servidor para que este actúe procesándola y respondiendo en consecuencia. El tratamiento adecuado de los formularios es una tarea de gran importancia que afecta de pleno a la seguridad de nuestras aplicaciones y que veremos más adelante.

Por ahora, debemos conocer qué tipo de información podemos enviar mediante esta herramienta y algunos elementos técnicos de HTML para un diseño correcto de formularios.

► Ejercicio 6: Formulario de registro de clientes en una web

En la aplicación del ejercicio anterior se necesita crear una página en su sitio web para que cualquier usuario pueda rellenar un formulario con sus datos personales.

Para ello debe crear un formulario paso a paso a lo largo de las siguientes secciones. En cada una de ellas se practicará con un tipo de control distinto. El resultado final serán tres documentos HTML:

- Un primer documento con lo indicado hasta la sección 5.11.
- Un segundo documento, creado a partir del anterior, para la sección 5.12.
- Un tercer documento para la sección 5.13.

5.1. Campos de texto y de envío

Posiblemente sean los controles de entrada de datos que se usan con más frecuencia. Cree una página web que contenga un formulario con tres campos de texto:

- Nombre. Campo de entrada de tipo `text`.
- Apellidos. Campo de entrada de tipo `text`.
- Clave. Campo de entrada de tipo `password`. A efectos de envío de datos funciona igual que el de tipo `text`. La diferencia estriba en que mientras se escribe en él, no se visualizan los caracteres por seguridad.

Incluya también un control de tipo `submit` para mostrar un botón de manera que al pulsarlo se envíe la información a un servidor web para que la procese. El aspecto final podría ser como el siguiente:

Registro de usuarios
Nombre: Apellidos: Clave de acceso:

Tenga en cuenta las siguientes consideraciones:

- El campo `action` del formulario debe dirigirse hacia un *script* que muestre los datos recibidos.
- La disposición de los elementos (todos en una misma línea) podría cambiar dependiendo de los *tags* que utilice.
- Asegúrese de usar correctamente los elementos de tipo `label`.

Para comprobar que la información del formulario se envía correctamente, deberá tener un *script* PHP en el servidor web que, simplemente, muestre los valores recibidos (ver figura). A continuación tiene un código que puede usar para este ejercicio. Deberá almacenarlo en un fichero llamado `procesar.php` en la misma carpeta que su documento HTML (la extensión `.php` es obligatoria) e incluir su referencia en el atributo `action` del formulario. A diferencia de los ficheros de tipo `.html`, que se pueden ver en cualquier navegador desde el sistema de ficheros local sin necesidad de tener instalado ningún servidor web, los programas PHP deben ejecutarse obligatoriamente desde un servidor web configurado convenientemente para ello.

<!DOCTYPE html>

```

<html>
<head>
  <meta charset="UTF-8">
  <title>Variables recibidas</title>
  <style type="text/css">
    .valor { color: red; }
  </style>
</head>
<body>
  <?php echo "<h1>Variables recibidas</h1>";
  showVar($_GET, 'Desde $_GET');
  showVar($_POST, 'Desde $_POST');
  function showVar($var, $msg) {
    echo "<h2>$msg</h2>";
    echo "<ul>";
    foreach ($var as $c => $v) {
      if (is_array($v)) {
        echo "<li>$c = <span class='valor'>";
        print_r($v);
        echo "</span></li>";
      } else
        echo "<li>$c = <span class='valor'>$v</span></li>";
    }
    echo "</ul>";
  }
  ?>
</body>
</html>

```

Variables recibidas

Desde \$_GET

- nombre = Javier
- apellidos = Martínez
- password = miclave secreta
- envio = Enviar datos

Desde \$_POST

Al no haber indicado nada al respecto, el formulario usa por defecto el método de envío GET. Puede mantenerlo así pero lo recomendable es que indique de forma expresa el método de envío con el atributo `method` del formulario (`method="GET"`).

Opcional

Puede probar otros atributos de los controles de entrada de tipo texto:

- **size**. Indica el tamaño de la caja de entrada de datos.
- **maxlength**. Indica el número máximo de caracteres que se pueden escribir en la caja.

5.2. Campos obligatorios

Continúe con el formulario y obligue a que el usuario introduzca un valor en el campo "nombre". Para ello debe usar el atributo **required**. Recuerde que esta validación la hace el navegador y que el nivel de soporte puede variar dependiendo del navegador y de su versión.

En ocasiones, aunque algunos campos se hayan etiquetado como obligatorios, es conveniente deshabilitar esas comprobaciones. Pruebe el uso del atributo **novalidate** en la etiqueta **form** para comprobarlo. Esto es particularmente útil durante el proceso de desarrollo para verificar que funciona correctamente la validación en el servidor, que se estudiará más adelante.

5.3. Información al usuario

Incluya un texto informativo en los campos de entrada para advertir al usuario de que el nombre de usuario es obligatorio, de que los apellidos no son obligatorios y de que introduzca una clave en el campo "clave". Utilice el atributo **placeholder**.

Compruebe que, aunque se incluya un texto en **placeholder**, este no se envía al servidor en caso de dejar vacío el campo.

5.4. Valor por defecto

Añada un nuevo campo de texto para indicar la nacionalidad. Dicho campo tendrá como valor por defecto "España". Para ello utilice el atributo **value**.

Observe la diferencia entre **placeholder** y **value**. Mientras que el primero tiene como finalidad informar al usuario (pero no enviar esa información al servidor), el segundo tiene como finalidad incluir información que puede ser enviada al servidor si no se modifica.

5.5. Campos especiales de texto

Añada un campo de tipo **email** para introducir el correo electrónico. Compruebe la validación que hace el navegador:

- Si se deja en blanco.
- Si se rellena con algo que no es una dirección de email válida.

¿Influye el navegador/versión en esta validación?

Opcional

En este caso, el navegador lo que hace es comprobar que el texto escrito tiene la forma de un correo electrónico y lo más probable es que lo haga mediante el uso de expresiones regulares. Puede incluir validaciones de este tipo en cualquier control de entrada usando el atributo **pattern**. Por ejemplo, puede incluir en los campos del nombre y de los apellidos un atributo **pattern** con una expresión regular que los valide solo si comienzan por una letra mayúscula.

5.6. Formato

En este punto, el aspecto del formulario podría ser similar al que se muestra a continuación (dependiendo de las etiquetas usadas):

Registro de usuarios

Nombre:
 Apellidos:
 Clave de acceso:

Nacionalidad:
 E-mail:

Aunque por ahora el aspecto del formulario no debe preocuparnos, para continuar el ejercicio es conveniente que los campos se muestren de forma un poco más legible ya que vamos a seguir añadiendo otros nuevos. En la siguiente figura se propone que cada campo de entrada se sitúe en una línea independiente.

Registro de usuarios

Nombre:
 Apellidos:
 Clave de acceso:
 Nacionalidad:
 E-mail:

Para ello puede usar varias alternativas:

1. Incluir los elementos **label** e **input**, que son de tipo **inline**, dentro de una etiqueta **p**, que es de tipo **block**.
2. Incluir alguna regla CSS para indicar que determinados elementos de tipo **inline** se comporten como elementos de tipo **block**.
3. Uso de **fieldset** para agrupar conjuntos de campos. Esto se puede combinar con las anteriores.

Modifique el formulario para que tenga un aspecto similar al de la figura.

En caso de que para cada entrada utilice un patrón de la forma `<label>Texto: <input ...> </label>` bastaría con incluir el siguiente código CSS dentro del tag `<head>`:

```
<style type="text/css">
  label { display:block; }
</style>
```

5.7. Menú desplegable

Incluya un campo con un menú desplegable para especificar el sexo del usuario al estilo del que se muestra en la figura. Mantenga el mismo orden de la figura e incluya como opción preestablecida la última de ellas.

Compruebe que el valor que se recibe en el servidor coincide con el texto de los ítems del menú.

Sexo:

Masculino
 Femenino
 No deseo responder

Añada un segundo desplegable para que el usuario elija una de las siguientes opciones:

- "Acepta el almacenamiento de mis datos y el envío a terceros".
- "Acepta el almacenamiento de mis datos pero no el envío a terceros".
- "No acepta el almacenamiento ni el envío de datos a terceros".

En casos como este en el que el texto de los ítems del menú es una cadena más o menos larga y arbitraria y que, incluso, podría modificarse una vez finalizada la aplicación web (piense, por ejemplo, en una web que se muestra en varios idiomas o en un proceso de revisión ortográfica independiente del de desarrollo) puede que no sea buena idea transmitir dicha cadena sino, en su lugar, algún valor que pueda usarse por nuestro código PHP/HTML y que sea transparente al usuario. Utilice el atributo `value` para que el valor enviado en cada una de las opciones sea uno de estos:

- TOTAL (para la primera opción).
- PARCIAL (para la segunda opción).
- NINGUNO (para la tercera opción).

5.8. Opciones excluyentes (*radio buttons*)

Añada un campo para que el usuario indique el idioma preferido para sus comunicaciones mediante *radio buttons* con un aspecto similar al de la figura.

A continuación realice las siguientes tareas:

1. Compruebe qué se envía si no se marca ninguna de las opciones.
2. Modifique el formulario para que esté marcada como opción por defecto una de ellas.

Asegúrese de usar correctamente la etiqueta `label` y verifique que su uso mejora la usabilidad del formulario.

Idioma para comunicaciones:

☐ Español

☒ Inglés

☐ Francés

5.9. Opciones múltiples (*checkbox*)

En casos en los que se desea poder marcar cero o más opciones de entre varias se utilizan los campos de tipo *checkbox*. Añada campos de entrada para que el usuario determine sus aficiones en una forma similar a la de la siguiente figura.

Ponga especial cuidado en el valor asignado al atributo `name` de la etiqueta `input`. Compruebe qué ocurre si:

1. Utiliza un valor como los que ha usado hasta ahora, por ejemplo `name="aficiones"`.
2. Utiliza un valor diferente de `name` para cada `input`.
3. Utiliza un valor con formato de *array*, por ejemplo `name="aficiones[]"`.

Aficiones:

☐ La música

☒ Leer libros

☐ El cine

☐ El deporte

Asegúrese de usar correctamente la etiqueta `label` y verifique que su uso mejora la usabilidad del formulario.

5.10. Soporte de navegadores a campos especiales

Hay varios tipos de campos de entrada introducidos en HTML 5 que no tienen igual soporte en todos los navegadores. Añada campos para introducir:

- Fecha de nacimiento (`date`).
- En qué semana se celebra Semana Santa este año (`week`).

Pruebe a visualizar el formulario en Firefox y en Chrome y visite la web <https://caniuse.com> para comprobar qué soporte tienen.

5.11. Campos ocultos

En ocasiones se necesita que desde el formulario se envíe información adicional a la introducida por el usuario en los distintos controles. Esto se consigue añadiendo un `input` de tipo `hidden`, que se comporta como un campo de texto normal pero que no se muestra.

Incluya un campo oculto que envíe un número de versión del formulario.

5.12. Campos con entrada deshabilitada

Para hacer este ejercicio cree un nuevo documento HTML que sea una copia del documento que lleva creado hasta el momento con el formulario. En este punto ambos documentos serán idénticos. Este ejercicio debe realizarlo sobre la copia manteniendo el documento original intacto para su evaluación.

Modifique el formulario para que los siguientes campos tengan valores por defecto ya introducidos y que no sean modificables:

- Nombre: Javier
- Apellidos: Martínez
- Clave: miclave
- Nacionalidad: España
- Email: javiermartinez@ugr.es
- Sexo: masculino
- Tratamiento de datos: TOTAL
- Idioma para comunicaciones: Inglés
- Aficiones: Leer libros, La música
- Fecha de nacimiento: 01/01/2000
- Semana Santa: 16 (semana 16)

Para ello estudie el uso de los atributos `readonly` y `disabled` y compruebe en qué casos se puede o debe usar cada uno y qué efecto tienen sobre el formulario y el envío.

¿En qué situaciones puede tener sentido que un campo que ha sido incluido en el formulario y que se muestra al usuario deshabilitado no se llegue a enviar al servidor?

5.13. Envío de ficheros

Para hacer este ejercicio cree un tercer documento HTML que sea una copia del documento con el formulario original. Este ejercicio debe realizarlo sobre la nueva copia manteniendo el documento con el formulario inicial intacto.

Añada un último campo para adjuntar una fotografía del usuario. El aspecto del formulario será

similar a este:

Registro de usuarios

Nombre:

Apellidos:

Clave de acceso:

Nacionalidad:

E-mail:

Sexo:

Tratamiento de datos:

Idioma para comunicaciones:

☐ Español

☒ Inglés

☐ Francés

Aficiones:

☐ La música

☒ Leer libros

☐ El cine

☐ El deporte

Fecha de nacimiento:

Semana Santa:

En este formulario, además de incluir el control para la entrada del fichero, debe modificar los atributos de la etiqueta `form`. Además, el *script* PHP que indique en el `action` debe incluir código para procesar el fichero (al menos para informar de que lo ha recibido correctamente). El siguiente código es una modificación del anterior que incluye esa funcionalidad. Puede almacenarlo en un fichero `procesar2.php` y utilizarlo de forma diferenciada en este último formulario:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Variables recibidas</title>
  <style type="text/css">
    .valor { color: red; }
  </style>
</head>
<body>
  <?php echo "<h1>Variables recibidas</h1>";
  showVar($_GET, 'Desde $_GET');
  showVar($_POST, 'Desde $_POST');
  showVarFiles();
  function showVar($var, $msg) {
    echo "<h2>$msg</h2>";
    echo "<ul>";
    foreach ($var as $c => $v) {
      if (is_array($v)) {
        echo "<li>$c = <span class='valor'>";
        print_r($v);
        echo "</span></li>";
      } else
        echo "<li>$c = <span class='valor'>$v</span></li>";
    }
    echo "</ul>";
  }
  function showVarFiles() {
    echo "<h2>Ficheros recibidos:</h2>";
    echo "<ul>";
```

```

foreach ($_FILES as $c => $v) {
    echo "<li>$c = <pre class='valor'>";
    print_r($v);
    echo "</pre>";
    if (is_uploaded_file($v['tmp_name']))
        if (is_image($v['tmp_name']))
            echo '";
        else
            echo '<p>El fichero no contiene una imagen</p>';
        echo "</li>";
    }
    echo "</ul>";
}

function is_image($file) {
    $datos = getimagesize($file);
    return $datos ? ((in_array($datos[2],
array(IMAGETYPE_GIF, IMAGETYPE_JPEG, IMAGETYPE_PNG, IMAGETYPE_BMP))) ?
true : false) : false;
}

?>
</body>
</html>

```

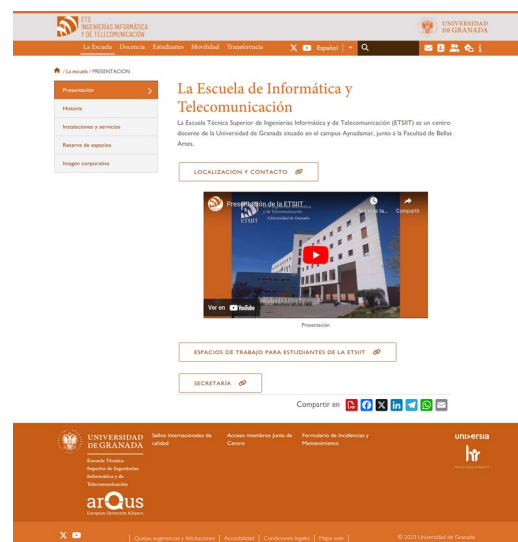
Observe que el *script* comprueba si el fichero subido es una imagen y, de ser así, en la respuesta muestra dicha imagen.

6. Sitios web

Rara vez, por no decir nunca, el desarrollo de un sitio web consiste en la creación de una única página web. Lo habitual es que un sitio web incluya multitud de páginas que, además, son creadas de forma dinámica en función de lo que desea ver el usuario y de información alojada en bases de datos del mismo u otros servidores.

Aunque un sitio web esté formado por muchas páginas web, es muy frecuente que todas esas páginas mantengan una estructura homogénea que dé al usuario sensación de coherencia. Es decir, la estructura general de todas las páginas es, esencialmente, la misma.

A modo de ejemplo puede visitar la web de la ETSIT (<https://etsiit.ugr.es>). Se muestran algunas capturas a continuación:



Puede ver que en cualquier página del sitio web se mantienen los siguientes elementos:

- Encabezado principal con el logotipo y nombre del Centro así como el logotipo de la UGR.
- Un menú de navegación principal que incluye una barra de búsqueda y enlaces a redes sociales u otros elementos de uso frecuente. El contenido del menú puede variar de una página a otra pero el aspecto y posición permanece.
- Un pie de página con enlaces a políticas de uso, indicaciones de accesibilidad, formularios de contacto, etc.
- En algunas páginas se incluye un menú de navegación lateral secundario.

► Ejercicio 7: Sitio web de reservas de aulas

En la web de la ETSIT puede ver en los accesos rápidos de la zona inferior uno que se denomina "Reserva de aulas". Este lleva a una aplicación web destinada a que los usuarios del sitio puedan solicitar el uso de las distintas aulas del Centro (para docencia, conferencias, etc.). En este ejercicio debe implementar las distintas páginas web de esa una aplicación para hacer las reservas de aulas de un centro docente. Concretamente debe disponer de las siguientes:

1. Página principal (index.html). Es una página de presentación del Centro e incluirá:
 1. Breve descripción del Centro.
 2. Alguna(s) fotografía(s).
 3. Algunos enlaces a eventos de interés.
2. Página con información sobre las aulas disponibles: puede ser una única página con toda la información o puede hacer una página para cada aula. Para cada aula debe incluir:
 1. Descripción.
 2. Capacidad.
 3. Algunas fotografías.
3. Página que muestra la lista de reservas hechas. Será una tabla con el siguiente aspecto:

Fecha	Horas	Sala	Usuario	Motivo
8/1/2025	9:30 - 11:30	Aula 0.2	Javier	TW
8/1/2025	10:30 - 11:30	Lab electrónica	Pedro	Examen de TOC
10/1/2025	13:30 - 14:30	Aula 0.3	Pedro	Conferencia

4. Página de contacto y registro que incluirá dos formularios:
 1. Uno para registrarse en el sitio como el que se ha hecho en ejercicios anteriores
 2. Otro para contactar con el autor del sitio que incluya: el email de quien está contactando, un título para la pregunta y que quiere hacer y otro para la pregunta.

A partir de esta información, lo primero que debería realizar es un boceto o prototipo simple. Esto se hace con varios objetivos en mente:

1. Visualizar la apariencia de la web. Aunque aún no conocemos CSS para hacer el maquetado de la web, nos ayudará a definir mejor lo que queremos construir.
2. Estructurar la información. Al construir la estructura de la información lo que estamos haciendo es jerarquizar los contenidos y, por lo tanto, obtener el anidamiento de los diferentes elementos HTML de cada página.
3. Detectar necesidades o carencias de la web. En el caso de este ejercicio esto puede no ser evidente por la simplicidad del mismo.
4. Disponer de un documento concreto que plasma las especificaciones que nos ha dado el cliente para hablar de nuevo con él y comprobar que la solución que estamos diseñando se corresponde con lo que él espera o para rectificar las especificaciones iniciales antes de seguir avanzando en el proyecto.

Deberá incluir un boceto para cada una de las páginas. El nivel de complejidad de este boceto se

deja a su elección (*wireframe*, *mockup*, ...) y deberá reflejar claramente cuál es la estructura común de todas las páginas del sitio web así como el contenido específico para cada una de ellas. Los bocetos se harán con alguna herramienta específica (Pencil, etc.) o de uso genérico (Libreoffice, Draw.io, etc.).

Una vez hechos los bocetos deberá implementar todas las páginas indicadas. En general, en este ejercicio deberá prestar especial atención al uso de *tags* HTML con la finalidad para la que han sido diseñados. En particular, debe cuidar aspectos como:

1. Uso correcto de los elementos estructurales propios de HTML5 (**header**, **footer**, **main**, **article**, **section**, ...) o genéricos (**div**).
2. Uso correcto de los encabezados de sección **H1**, **H2**, **H3**, ...
3. Agrupamiento correcto de elementos HTML. No hay una única forma de hacerlo pero unas serán mejores que otras: intente que la suya sea óptima.
4. Uso de elementos tales como:
 1. Imágenes
 2. Listas
 3. Otros estudiados en clase.

Aunque haya hecho unos bocetos que simulen el aspecto final de su aplicación (uso de imágenes, tipografías, colores, ubicación, etc.) debe tener en cuenta que en este ejercicio no puede hacer uso de CSS en ningún caso y centrarse exclusivamente en la realización de documentos HTML. Tampoco puede usar tablas HTML para conseguir dar determinado aspecto a la web.

7. Validación de documentos HTML

Llegados a este punto es más que probable que haya conseguido realizar los ejercicios planteados dado que su dificultad es mínima. Sin embargo ¿se han realizado correctamente? No hay una forma única de escribir una página web pero lo que debería tener presente siempre es que cualquier trabajo profesional que realice debería seguir estándares internacionales para asegurar un buen mantenimiento, eficacia, portabilidad, etc.

En el caso de los documentos HTML deberíamos preguntarnos, fundamentalmente, dos cosas:

1. ¿Está bien estructurada la información?
2. ¿El documento se ajusta al estándar actual de HTML?

Para responder a la primera pregunta se han dado algunas pautas en la asignatura, aunque algunas pueden ser discutibles según distintos profesionales. Esto lo irá aprendiendo conforme aumente sus conocimientos en la materia.

La segunda pregunta tiene una respuesta mejor definida y una forma sencilla de verificarlo es haciendo uso de algún servicio de validación automática. El propio consorcio W3C mantiene una web *online* que puede validar documentos HTML y mostrar los errores detectados.

► Ejercicio 8: Validación de documentos HTML

Acceda a la web <https://validator.w3.org/> y compruebe que los ejercicios que ha realizado siguen los estándares.

Como está aprendiendo aún, es probable que tenga algunos fallos. En este ejercicio deberá validar las siguientes páginas:

- Formulario de la sección 5.13. Mantenga su ejercicio original (con errores) e incluya un listado con los errores detectados por el validador. A continuación, cree un duplicado del ejercicio y corrija los errores.
- Páginas del ejercicio de la sección 6. Mantenga los originales, incluya un listado con los errores y cree una nueva web con los errores corregidos.

8. Entrega de la práctica

El profesor dará instrucciones adicionales sobre lo que hay que incluir en la entrega de esta práctica así como los plazos para ello. En cualquier caso, el material deberá subirse al servidor web de la asignatura (void.ugr.es) y se evaluará únicamente el material subido al servidor.

Para esta práctica en concreto:

- Dentro de su carpeta **public_html** debe incluir una carpeta llamada "**practicaHTML**".
- Dentro de esa carpeta debe incluir todos los ficheros requeridos. Puede usar subdirectorios si lo estima oportuno.
- Debe existir un fichero llamado **index.html** con enlaces, ordenados, a todos y cada uno de los ejercicios realizados.
- Debe incluir también los *scripts* PHP para la recepción de formularios.

De esta forma, para acceder al sitio web que ha creado se usará la URL:

`https://void.ugr.es/~USUARIO/practicaHTML`

Observaciones:

- No cumplir con alguno de los requisitos de entrega invalidará la entrega completa.
- No se admitirán entregas por ningún otro medio.

Recomendación: no deje para el último momento la subida de la práctica a void.ugr.es. Aunque la práctica es sencilla podría haber algo que no funcione y puede que no tenga tiempo de repararlo.